# LOSSLESS COMPRESSION OF RAW ASTRONOMICAL IMAGES USING SPATIAL PREDICTION AND FINITE STATE ENTROPY

**Tomás Brás**
University of Aveiro
112665
tomas.bras@ua.pt

**David Pelicano**
University of Aveiro
113391
davidpoetapelicano@ua.pt

**Afonso Ferreira**
University of Aveiro
113480
afonso.ferreira@ua.pt

**ABSTRACT**

We present three lossless compressors for raw 16-bit astronomical images ($1500 \times 1500$ pixels, big-endian), each targeting a distinct point on the speed–ratio trade-off. **prism-block** uses a gradient-adjusted predictor (GAP) and a context-adaptive range coder across parallel horizontal bands, reaching 3.51 bpp in $\sim$81 ms. **HAIS** selects the best of five per-block predictors — including training-free least-squares with up to five spatial taps — codes residuals with Finite State Entropy (tANS) [2], and optionally splits the low-byte stream by the high-byte context, achieving 3.40 bpp in $\sim$187 ms. A third compression-focused codec extends HAIS with a wider predictor context and a context-aware mode-selection cost; results are pending. Both completed codecs outperform the best generic compressor (zstd-19, 4.00 bpp) by at least 0.49 bpp while remaining faster than bzip2-9 and xz-9, and they occupy non-dominated positions on the benchmark Pareto frontier.

## 1 Introduction

Raw images produced by ground-based astronomical telescopes are large, structured numerical arrays. Each image in this study is a $1500 \times 1500$ raster of unsigned 16-bit pixels stored in big-endian order, occupying 4.5 MB. These arrays differ from conventional files in three important ways. First, adjacent pixels are spatially correlated: the sky background and optical point-spread function create smooth gradients that extend over many pixels. Second, the readout electronics add a constant *bias pedestal* to every pixel, introducing a strong DC component whose removal collapses the residual distribution around zero. Third, the data are 16-bit integers, whereas most general-purpose compressors operate on byte streams, missing inter-byte structure.

These properties motivate *predictive* compression: subtract a spatial prediction $\hat{p}$ from each pixel, code only the residual $r = p - \hat{p}$, and exploit the fact that a good predictor concentrates the residuals near zero — thereby reducing entropy and improving compression.

We present three lossless compressors that exploit these properties, each targeting a distinct point on the speed–ratio trade-off:

- **prism-block** applies a gradient-adjusted predictor (GAP) to the full 16-bit value and codes residuals with a range coder driven by eight context-conditioned adaptive models across parallel horizontal bands. It prioritises throughput ($\sim$81 ms per image).
- **HAIS** divides the image into square blocks, selects the best of five candidate predictors per block — including training-free least-squares — splits residuals into independent byte streams, and codes them with Finite State Entropy (FSE/tANS) [2]. It balances speed and ratio ($\sim$187 ms, 3.40 bpp).
- **[Third codec]** extends the HAIS framework with a wider predictor context and a cost function that directly models the conditional entropy of the two byte streams. It prioritises compression ratio at the cost of extra computation. (Section 2.3.)

Section 2 describes the three codecs in detail. Section 3 defines the experimental protocol. Section 4 reports compression ratios and runtimes. Section 5 draws conclusions.

## 2 Codec Design

All three codecs share the same high-level pipeline: parse the raw big-endian 16-bit image; apply a spatial predictor $\hat{p}$; zigzag-encode the signed residual $r = \text{pixel} - \hat{p}$ as an unsigned symbol $z = 2r$ if $r \geq 0$, $z = -2r - 1$ otherwise; entropy-code the result. Zigzag maps small-magnitude residuals — which are the common case after a good prediction — to small unsigned integers, concentrating the symbol distribution near zero regardless of sign.

### 2.1 prism-block: Speed-Focused

**Partitioning and parallelism.** The image is split into horizontal bands of 150 rows each (10 bands for $1500 \times 1500$). Bands are compressed independently; a shared `std::atomic<int>` counter distributes bands to threads via `fetch_add`, achieving lock-free load balancing.

**Predictor.** Each pixel is predicted by the *Gradient-Adjusted Prediction* (GAP) function of its three decoded neighbours: left ($W$), above ($N$), and above-left ($NW$). Horizontal and vertical gradients are estimated as $d_h = |W - NW|$ and $d_v = |N - NW|$. When one gradient dominates ($d_h > 2d_v$ or $d_v > 2d_h$), the perpendicular neighbour is used directly; otherwise a gradient-weighted blend is computed and clamped:

$$\hat{p} = \mathrm{clamp}\left(\frac{(d_v + 1)\,W + (d_h + 1)\,N}{d_v + d_h + 2},\ \min(W, N),\ \max(W, N)\right).$$

This predictor handles edges and flat regions without a mode switch.

**Entropy coding.** After zigzag encoding, each 16-bit symbol $z$ is split into $z_h = z \gg 8$ (high byte) and $z_l = z \,\&\, 0\text{xFF}$ (low byte). Both bytes are coded with a carry-based range coder backed by adaptive Fenwick-tree models. $z_h$ uses a single shared model. $z_l$ is conditioned on the magnitude of the two preceding residuals:

$$c = \mathrm{bin}(|\hat{r}_W| + |\hat{r}_N|),$$

yielding eight context models whose bin boundaries are $\{0\}, \{1\text{–}2\}, \{3\text{–}8\}, \{9\text{–}32\}, \{33\text{–}64\}, \{65\text{–}128\}, \{129\text{–}512\}, \{> 512\}$. Each model halves all counts when the running total reaches $2^{16}$, tracking distribution shifts adaptively within a band.

### 2.2 HAIS: Balanced

**Dimension Inference and Block Selection**

HAIS accepts optional width/height arguments; otherwise it infers dimensions from the file size by trying the integer square root first, then a list of common astronomical sensor widths (1500, 2048, 4096, ...). It then selects the largest block side $b \leq 512$ that divides both $W$ and $H$ exactly and yields at least four blocks. For $1500 \times 1500$ this gives $b = 500$, producing a $3 \times 3$ grid of nine blocks of $500 \times 500$ pixels.

**Predictive Modes**

For each block, HAIS evaluates five candidate predictors and retains the one with the lowest estimated coding cost (defined below).

**Mode 0 (raw)** Symbols are raw pixel values; no prediction.

**Mode 1 (avg)** $\hat{p} = \lfloor (W + N + NW + 1)/3 \rfloor$.

**Mode 3 (gmean)** $\hat{p} = \bar{g}$, the global image mean computed once before blocking. Removes the bias pedestal in all frames.

**Mode 4 (LS-4)** OLS with features $(W, N, NW, 1)$ per block via Gauss-Jordan; four float32 weights (16 B) stored in the block header.

**Mode 6 (LS-5)** Extends Mode 4 with $NE$ and $NN$; five weights (20 B). Both LS modes use the block's own pixels — no external training.

The mode selection criterion penalises stored weights:

$$\mathrm{cost}_m = H(\mathrm{hi}_8) + H(\mathrm{lo}_8) + \frac{8\,\delta_m}{n_{\mathrm{pix}}},$$

where $H(\cdot)$ is empirical byte entropy and $\delta_m \in \{0, 0, 0, 16, 20\}$ B is the weight header size.

**Entropy Coding: FSE/tANS**

Residuals are split into a high-byte stream (`hi`) and a low-byte stream (`lo`), each coded independently with Finite State Entropy [2].

The FSE table has SCALE $= 2^{16} = 65536$ states. Raw symbol counts are normalised to sum to SCALE (with a minimum frequency of 1 for observed symbols) and spread deterministically using Collet's step formula: step $=$ (SCALE $\gg$ 1) + (SCALE $\gg$ 3) + 3. The encoder processes symbols in reverse, emitting variable-length transition bits stored in a `BitWriter`; the decoder reads the two-byte final state and reconstructs symbols in forward order.

The frequency table is serialised in one of three compact formats: *single-symbol* (flag `0x01`, 2 B total) when all values are identical; *sparse* (flag byte $=$ nnz $\in [2, 254]$, followed by nnz $\times$ 5 B symbol–frequency pairs); or *dense* (flag `0x00`, $256 \times 4$ B) when all 255–256 symbols appear.

**Context FSE**

Because the high byte of a small residual is almost always zero, the lo8 distribution differs substantially depending on whether $\mathrm{hi} = 0$. When the fraction of zero high bytes falls between 2% and 98%, HAIS builds two conditional lo8 streams — `lo_zero` (pixels with $\mathrm{hi} = 0$) and `lo_nonzero` — and uses the three-stream layout only if its compressed size is strictly smaller than the two-stream baseline. The high-byte stream is encoded once and shared between both candidate payloads. Bit 7 of the mode byte signals the active layout.

**Parallel Execution**

Blocks are dispatched to a thread pool via a single `std::atomic<int>` counter with `fetch_add`, writing results into pre-allocated per-block slots with no mutex required.

### 2.3 [Third Codec]: Compression-Focused

## 3 Experimental Setup

### 3.1 Dataset

The benchmark corpus consists of 8 raw astronomical images, each a $1500 \times 1500$ raster of unsigned 16-bit big-endian pixels (4.5 MB per file, $2.25 \times 10^6$ pixels). The images were acquired by different ground-based telescopes (VLT, TJO, GTC, INT, JKT, LCO, WHT) and span a deliberate variety of image types: bias frames (near-constant DC pedestal), flat fields (smooth illumination gradients), and science frames (stellar fields, galaxies). This diversity ensures that no single predictor type dominates the results artificially.

### 3.2 Reference Compressors

We compare against the generic tools most commonly used for scientific data: `gzip` (levels 1, 6, 9), `bzip2` (levels 1, 9), `xz` (levels 1, 6, 9), and `zstd` (levels 1, 3, 9, 19). All are invoked on the raw byte stream without any pre-processing or endian conversion. Results from two repository-specific codecs (`prism` and `helix`) are included for context.

### 3.3 Evaluation Protocol

The primary metric is bits per byte of the original file:

$$\text{bpp} = \frac{8 \times \text{compressed bytes}}{\text{original bytes}}.$$

For a 16-bit image, bpp $=$ bits per pixel$/2$. Compression and decompression times are measured as wall-clock elapsed time for a single run on a single machine; all compressors run sequentially in the same benchmark process to avoid contention. Losslessness is verified by byte-exact comparison of the decompressed output against the original file; any mismatch disqualifies the result. Binary size constraints from the project specification are satisfied: compress $\leq 5$ MB, decompress $\leq 1$ MB.

## 4 Results

### 4.1 Compression Ratio

Table 1 reports bits per byte for each of the eight benchmark images. Both completed domain-specific codecs outperform every generic compressor on every image. HAIS achieves the best ratio overall, reducing average bpp by $0.60$ versus zstd-19 ($-15\%$) and by $0.18$ versus bzip2-9. prism-block follows closely ($+0.11$ bpp over HAIS), still beating every generic tool. Image F (science frame with bright sources and strong gradients, where LS modes are selected) is the hardest for all codecs; image G (bias frame with near-constant signal) is the easiest.

Table 1: Compression ratio in bits per byte (bpp). Lower is better. "—" denotes a codec not yet benchmarked.

| Image | prism-block | HAIS | [Third] | zstd-19 | bzip2-9 |
|---|---|---|---|---|---|
| A | 4.44 | 4.39 | — | 5.46 | 4.62 |
| B | 3.30 | 3.24 | — | 3.77 | 3.40 |
| C | 2.60 | 2.43 | — | 2.74 | 2.59 |
| D | 2.85 | 2.73 | — | 3.16 | 2.89 |
| E | 2.73 | 2.56 | — | 2.91 | 2.71 |
| F | 6.20 | 6.14 | — | 7.06 | 6.47 |
| G | 2.55 | 2.41 | — | 2.71 | 2.54 |
| H | 3.41 | 3.29 | — | 4.20 | 3.41 |
| **Mean** | **3.51** | **3.40** | — | 4.00 | 3.58 |

### 4.2 Speed

Table 2 shows average per-image runtimes. prism-block achieves 81 ms total, comparable to zstd-3 (84 ms) but with 0.85 bpp better compression. HAIS takes 187 ms — 2.3× slower than prism-block, but 3.4× faster than bzip2-9 and 14× faster than xz-9, while achieving equal or better compression in every case. Note that zstd-1 (not shown) achieves ∼42 ms at 4.49 bpp; prism-block trades ∼2× more time for a 0.98 bpp gain.

Table 2: Average per-image runtimes (ms), single wall-clock run. "—" denotes a codec not yet benchmarked.

| Compressor | Compress | Decompress | Total | bpp |
|---|---|---|---|---|
| prism-block | 35 | 46 | 81 | 3.51 |
| HAIS | 89 | 98 | 187 | 3.40 |
| [Third] | — | — | — | — |
| zstd-3 | 66 | 18 | 84 | 4.32 |
| bzip2-9 | 394 | 233 | 628 | 3.58 |
| xz-9 | 2503 | 156 | 2659 | 3.69 |
| zstd-19 | 2487 | 25 | 2512 | 4.00 |

### 4.3 Pareto Analysis

Figure 1 plots all benchmarked compressors in the bpp-vs-time plane (log-scale time axis). The Pareto frontier — the set of compressors for which no other codec achieves simultaneously lower bpp *and* lower runtime — passes through prism-block and HAIS among the completed codecs. prism-block is non-dominated: it is faster than every generic compressor except zstd-1/3 while achieving substantially better compression than any of them. HAIS reaches the lowest bpp of any benchmarked compressor while remaining faster than bzip2-9, xz, and zstd-19. The third codec is expected to improve on HAIS's compression ratio at the cost of additional runtime, extending the frontier further left and down.

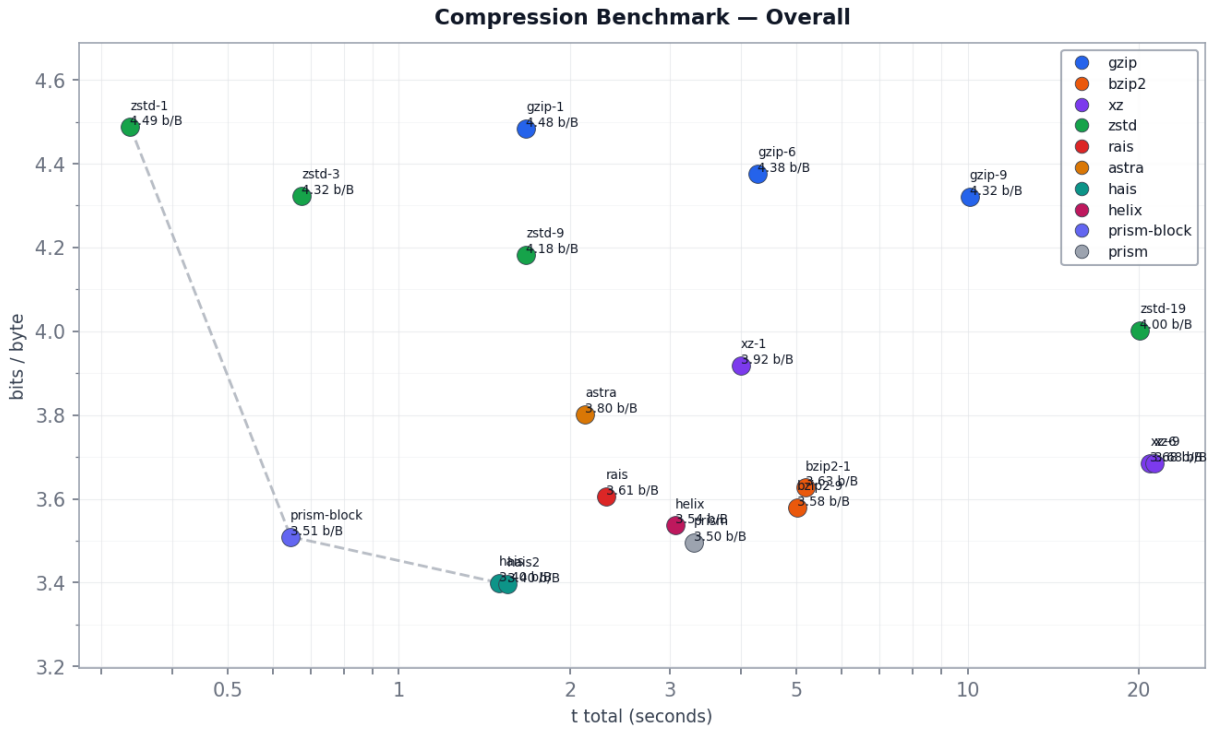


Figure 1: Compression ratio (bpp) vs. total runtime (compress + decompress, log scale) for all benchmarked compressors. Lower-left is better. Dashed curve: Pareto frontier over completed codecs.

## 5 Discussion and Conclusion

**Why domain-specific codecs win.** Generic compressors operate on byte streams and therefore see the two bytes of each 16-bit pixel as unrelated. This breaks the spatial correlation that spans pixel boundaries in big-endian encoding. Our codecs operate on 16-bit values directly, predict each pixel from its decoded neighbours, and entropy-code only the

residual. The result is a reduction of $0.49$–$0.60$ bpp over zstd-19 depending on the codec, despite zstd-19 using a much slower, high-effort general-purpose optimiser.

**Role of the predictor.** Mode selection on the eight benchmark images reveals that Mode 3 (global mean subtraction) is the most frequently chosen predictor, dominating on bias frames and flat fields where a near-constant DC offset accounts for most of the signal (images D, E, G, H). Mode 1 (three-neighbour average) prevails in smooth stellar backgrounds (image A). LS modes are selected only in image F, which contains bright point sources and extended spatial gradients that exceed the three-neighbour context. This pattern confirms that the bias pedestal is the single most compressible feature of astronomical frames; the LS predictor adds value only when the residual spatial structure is non-trivial after DC removal.

**Entropy coding design.** Splitting 16-bit residuals into independent hi8 and lo8 streams is the key structural choice. After a good prediction, hi8 is near-uniform zero, yielding a near-zero-entropy stream (often a 2-byte single-symbol FSE packet); all useful variation is carried by lo8. The context FSE variant conditions lo8 on whether hi8 is zero, exploiting the bimodal structure of the lo8 distribution. prism-block achieves the same effect via eight adaptive context models without transmitting a frequency table, trading $2.3\times$ speed for $0.11$ bpp less compression compared to HAIS.

**Limitations.** The corpus covers only 8 images from a single training set; rankings may shift on sensors with different noise characteristics or gain settings. Runtime measurements reflect a single wall-clock run on one machine and include process-launch overhead for the generic tools. The dimension-inference heuristic in HAIS may fail for non-standard sensor sizes; explicit width/height arguments bypass this limitation.

**Conclusion.** Specialised predictive compression outperforms general-purpose tools on raw astronomical imagery. HAIS achieves 3.40 bpp versus 4.00 bpp for zstd-19, a 15% reduction, while remaining $14\times$ faster than xz-9. The dominant gain comes from the predictor: global mean removal alone handles most calibration frames; least-squares adaptation provides further gains on science images. The entropy coder contributes relatively little once residuals are well predicted. Completing and benchmarking the third codec is the immediate next step.

## References

[1] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
[2] Yann Collet. Finite State Entropy — a new entropy coder. `https://github.com/Cyan4973/FiniteStateEntropy`, 2013.
[3] Yann Collet. Zstandard — fast real-time compression algorithm. `https://github.com/facebook/zstd`, 2016.
[4] William D. Pence, L. Seaman, R. Seaman, and R. White. FITS lossless image compression. *Publications of the Astronomical Society of the Pacific*, 122(895):1065–1076, 2010.
[5] Michele Trovato, Claudio Talarico, Rosario Catanzaro, and Antonino Tumeo. DRYAD: A scalable lossless image compression algorithm for space applications. In *Proc. Data Compression Conference (DCC)*, 2018.